

# OPERATION IRON VEIN: A Sealed XChaCha20-Poly1305 Valve Command Path, a Logic-Bomb Implant, and Anti-Debug Analysis on an RP2350 SCADA Controller

Kevin Thomas  
George Mason University  
Fairfax, VA, USA  
kthoma60@gmu.edu

**Abstract:** A healthy controller can still be a hostile one. The OPERATION IRON VEIN build is a bare-metal RP2350 SCADA valve controller and its companion gateway, and it is Act III of the OPERATION COLD IRON story. The node reads an SG90 servo as the pipeline valve, a VS1838B infrared receiver as an operator remote for OPEN, CLOSE, and ESTOP, a DHT11 as the process sensor, a 1602 LCD as the SCADA status and alarm readout, red/yellow/green LEDs as the VALVE FAULT, COMMAND PENDING, and VALVE NOMINAL annunciator, a debounced button as the emergency stop, and an RYLR998 LoRa link to a sealed command gateway. Every request and command is sealed end to end with XChaCha20-Poly1305 (RFC 8439 ChaCha20 and Poly1305 with an HChaCha20 subkey) keyed through Argon2id (RFC 9106, profile t=3, p=1, m=64 blocks), implemented in-repo with no third-party code and tested against published vectors. Act III adds a malware track to the vulnerability-only foundation of Acts I and II: a benign FROSTLINE implant, compiled only under a SANDBOX\_ONLY guard, that beacons on the DE AD BE EF magic every eight ticks, arms a logic bomb on the eight-byte magic FROSTLINE, closes the valve three ticks later independent of the operator, reads CoreDebug DHCSR at 0xE00EDF0 to suppress itself while a debug probe is attached, and writes a one-time 0xC7 marker into the reserved sector 0x103FF000 as a preview of persistence. We document the peripheral set, the wire and envelope formats, the sealed command path with its anti-replay window and authenticated state tag, the blue-half controls (sealed command path, ESTOP priority, fail-closed policy, build integrity), the malware design, and an honest threat model that names the shared lab key, the open debug port, and the deliberately inert implant as explicit decisions rather than accidents. A 132-case, 409-check native suite reaches 100% line coverage of every owned firmware module.

**Index Terms:** RP2350, SCADA, pipeline valve, XChaCha20-Poly1305, Argon2id, anti-replay, authenticated state, malware analysis, logic bomb, beacon, anti-debug, CoreDebug DHCSR, persistence, fail-closed, embedded firmware.

## I. Introduction

Act I of the OPERATION COLD IRON story was the silent lie: a cold-chain monitor that reported minus eighteen degrees while the store warmed. Act II was the door: an access gate that kept its final verdict in plain SRAM while the cryptography around it was correct. Act III is the payload that is already inside. The frame pulled from the gate carried a route, and the route ended at a NorthPharma pipeline. The valve controller

is healthy. You can read it, and nothing looks broken. That is the horror. It is not buggy, it is weaponized. A hidden implant beacons over LoRa, and a logic bomb will slam the valve shut at a trigger and spike the line pressure. NorthPharma is the Ministry's front; FROSTLINE planted the implant.

The reveal that ties the three acts together is the shift in what a defect is. Act I was a lie about a number. Act II was a lie about a person. Act III is a machine that works

exactly as designed and still does harm, because a second party shares the chip. The three-act arc is therefore a widening of the trust boundary: first the sensor, then the state, and now the firmware image itself.

A pipeline valve is a simple machine. A process sensor reports the line, an operator requests an open or a close, an actuator moves the valve, and an annunciator says whether the line is nominal. Three properties must hold at once: integrity, so the command that reaches the valve is the authorized one; availability, so the valve is there when the process needs it; and state, so the controller does not trust a stale or tampered verdict. The naive controller collapses all three. Act III both fixes that and then goes further, because the first act with a malware track has to answer a question a protocol cannot: what happens when the attacker already runs on the device?

The classroom goal is to teach both halves. The red half and the malware track find the defects: forge a command, replay a captured command, trigger the logic bomb, find the beacon, and step past the anti-debug trap. The blue half and the fix track seal the controller: a sealed, guarded command path, a monotonic anti-replay window, an authenticated state tag, an absolute emergency-stop priority, a fail-closed policy, and build-level integrity. The centerpiece is a lesson about scope. The wire is authenticated, the verdict is tagged, and the implant still owns the actuator, because the implant never needed the wire.

### I.I. Contributions

This paper provides the following concrete contributions:

- A bare-metal RP2350 SCADA valve controller that drives the full Embedded Hacking peripheral set: an SG90 valve actuator, a VS1838B NEC operator remote with OPEN, CLOSE, and ESTOP commands, a DHT11 process sensor, a 1602 LCD SCADA readout over I2C, a red/yellow/green annunciator, a debounced emergency stop, and RYLR998 command and audit with a declared-length payload parser.
- A sealed valve command path with a guarded command set, a monotonic anti-replay sequence window, and an authenticated state tag that detects a debugger-written verdict before the actuator moves.
- An in-repo, third-party-free cryptographic layer: Argon2id key derivation (RFC 9106) and

XChaCha20-Poly1305 authenticated encryption (RFC 8439 with an HChaCha20 subkey), sealed per frame into a lowercase hex envelope with the valve node identifier bound as associated data.

- A benign FROSTLINE implant, confined to a `SANDBOX_ONLY` build, that demonstrates a covert LoRa beacon, an arming-trigger logic bomb that closes the valve, a CoreDebug `DHCSR` anti-debug trap, and a one-time reserved-sector persistence marker.
- A SCADA gateway that authenticates before it parses, logs authenticated and rejected requests distinctly, and answers only authenticated requests with a sealed command, plus a spoofing client whose forged and replayed commands are rejected.
- A corpus-aligned packet artifact contract (`packet_artifact.json` / `packet_artifact.h`) with a build-time staleness guardrail.
- A 132-case, 409-check native test suite reaching 100% line coverage of every owned firmware module, including the implant, checked against published RFC test vectors.
- A threat model that states explicitly what the lab profile does and does not protect, and an honest account of the implant as a sanitized educational artifact.

## II. Related Work

Industrial control security is a mature field, and the SCADA valve is a canonical target. The DHT11 one-wire sensor [1] and the NEC infrared remote encoding [2] are broadly documented and representative of the process and operator surfaces real installations deploy. The authenticated construction we use follows the ChaCha20-Poly1305 standard [6], and Argon2 follows the Argon2 specification [7]. The stateful construction is the standard replay defense found in secure-messaging and payment protocols, applied here at the scale of one valve.

Two lines of work frame Act III specifically. The first is the long line of firmware implants and logic bombs [8]: code that lives on the device, waits for a trigger, and acts through the device’s own actuators rather than through its protocol. The second is anti-analysis, in which a payload inspects the system to decide whether it is being observed and changes behavior accordingly. The CoreDebug `DHCSR` check used here is a minimal, well-

known example of that technique, chosen because it is legible on a debug probe and cheap to verify.

The pedagogical use of intentionally vulnerable firmware is established [5]. The difference in this act is that the vulnerable artifact is not a broken protocol but a hostile module that coexists with a correct one. The exercise demonstrates both the malware and its removal on the same board, and it makes the scope limit explicit: an authenticated wire does not authenticate the machine under it.

### III. System Model

The system consists of four roles:

- **Valve controller (RP2350 firmware):** decodes the infrared operator remote, verifies and authorizes sealed gateway commands, annunciates COMMAND PENDING, checks the DHT11 process band, drives the servo valve, enforces the emergency stop, and renders the SCADA readout.
- **SCADA gateway (gateway):** listens on the instructor serial port, authenticates and logs every +RCV frame to `valve_log.csv`, decides authorization, and answers an authenticated request with a sealed valve command carrying a monotonic sequence number and an authenticated state tag.
- **Edge simulator:** a laptop process that behaves like an additional controller, sealing requests with the same field key.
- **Attacker:** a laptop process that claims the gateway address, forges a command, or replays a captured command at the controller.

Let  $A \in \{0,1\}^{16}$  be the LoRa node address,  $L$  the declared payload byte length, and  $C$  the ASCII payload, which is a lowercase hex envelope. The unauthenticated wire framing is:

$$+RCV=, A, ,, L, ,, C, ,, rssi, ,, snr, CRLF$$

Because the hex payload has no commas, the framing is simpler than Act I's comma-bearing JSON, but the receiver still slices by declared length rather than by counting delimiters, for exactly the reason Act I documents.

#### III.I. Hardware Configuration

The classroom node is a Pico 2 (RP2350) carrying the full Embedded Hacking kit. The pin map is identical to Acts I and II so one breadboard serves all three, and it is fixed in `include/pipeline_valve.h` and enforced by the native test suite:

| Signal                             | RP2350 GPIO |
|------------------------------------|-------------|
| DHT11 process sensor (one-wire)    | GP4         |
| 1602 LCD SDA (I2C1)                | GP2         |
| 1602 LCD SCL (I2C1)                | GP3         |
| RYLR998 RX (UART1 TX)              | GP8         |
| RYLR998 TX (UART1 RX)              | GP9         |
| Infrared operator remote (VS1838B) | GP5         |
| Valve actuator (SG90 PWM)          | GP14        |
| Emergency stop button              | GP15        |
| Red VALVE FAULT LED                | GP16        |
| Yellow COMMAND PENDING LED         | GP17        |
| Green VALVE NOMINAL LED            | GP18        |
| Onboard heartbeat LED              | GP25        |

The LCD backpack uses the PCF8574 at 7-bit address `0x27`. The servo runs from a 50 Hz PWM output with a 1000 uF bulk capacitor on the 5 V rail to absorb the stall current when the valve moves; seated is 0 degrees and open is 90 degrees. At boot the controller programs its own transceiver (`AT+ADDRESS=7, AT+NETWORKID=18`) and the gateway programs the receiver (`AT+ADDRESS=1, AT+NETWORKID=18`) before logging, so command traffic is only delivered between radios that share the network identifier.

The valve is fail-closed: it is driven to the seated position at initialization and on every failure path, so loss of power, a failed process read, a malformed command, a tampered verdict, or a lost link all leave the valve closed. The emergency stop is the highest-priority input and forces the same safe state.

#### III.II. Operator Remote, Process Sensor, and Annunciation

The VS1838B is a 38 kHz demodulating infrared receiver whose output idles high and pulls low during a mark. The decoder times edges and reconstructs a NEC pulse

train, then feeds the command into the guarded operator set. `MONITOR_IR_OPEN` is `0x01`, `MONITOR_IR_CLOSE` is `0x02`, and `MONITOR_IR_ESTOP` is `0x03`. The optical surface has no key and no challenge, so an operator command is treated as a request, not as an authorization; the sealed radio path is what moves the valve in the defended design, and the optical path is a surface the red half examines.

The DHT11 is the process sensor. A reading that fails its checksum is never safe, and a valid reading outside the safe band (`VALVE_TEMP_MIN_TENTHS` = `-50` to `VALVE_TEMP_MAX_TENTHS` = `100`, that is `-5.0 C` to `10.0 C`) is out of band. Either case marks the process as not nominal, so a dead or unplugged sensor, or a genuinely unsafe line, is visible in the SCADA readout.

Exactly one status lamp is lit at a time. Red is `VALVE FAULT`, yellow is `COMMAND PENDING` while the valve travels, and green is `VALVE NOMINAL`. The 1602 LCD shows the valve state on line one (`ST:OPEN`, `ST:CLOSED`, `ST:MOVING`, `ST:FAULT`) and the process and link verdict on line two (`PROC:OK LNK:UP`).

## IV. Wire Protocol

The operator or the edge simulator seals a one-byte valve command into an XChaCha20-Poly1305 envelope and sends it to the gateway:

```
AT+SEND=0001,82,<82 lowercase hex characters>
```

The gateway answers an authenticated request with a sealed valve command. The command plaintext is a 21-byte body:

```
seq[4] (little-endian) || cmd[1] || state_tag[16]
```

where `seq` is the monotonic gateway sequence number, `cmd` is `0x00` for close or `0x01` for open, and `state_tag` is a tag over the authorization record the command would produce. The gateway sends the reply back to the claimed sender:

```
AT+SEND=<node>,122,<122 lowercase hex characters>
```

The radio's AT command buffer (`RADIO_AT_CMD_MAX_LEN`), the inbound +RCV buffer (`RADIO_RCV_MAX_LEN`), and the generated artifact limit (`PACKET_MAX_RCV_LEN`) are all 256 bytes, which comfortably holds the largest possible envelope plus framing.

The line accumulator is one byte larger than the command limit so it can hold the terminating NUL.

The firmware enforces a guarded command set in `control_parse`: the recovered command byte is rejected when it is greater than `VALVE_COMMAND_OPEN` (one). This is the sealed replacement for the unauthenticated valve angle injection, and it means a raw angle or an out-of-set value can never reach the actuator.

### IV.I. Envelope on the Wire

The sealed envelope is the lowercase hexadecimal encoding of a fixed layout:

```
nonce[24] || ciphertext[L] || tag[16]
```

For a one-byte request body this is  $24 + 1 + 16 = 41$  bytes, or 82 hex characters. For a 21-byte command body this is  $24 + 21 + 16 = 61$  bytes, or 122 hex characters. The maximum plaintext is 48 bytes (`ENVELOPE_MAX_PLAINTEXT`), so the largest possible envelope is  $24 + 48 + 16 = 88$  bytes, or 176 hex characters plus a trailing NUL, for a 177-byte envelope buffer (`ENVELOPE_MAX_HEX_LEN`). The declared length  $L$  in the framing is the length of the hex string, not of the underlying plaintext.

### IV.II. Declared-Length Slicing Invariant

Given the substring  $T$  after the second comma:

$$C = T[0 : L] \quad \text{and} \quad T[L] = ,$$

The invariant  $T[L] = ,$  is checked, so a mismatch between the declared length and the actual payload is a parse error rather than silent corruption. This is the same discipline Act I adopts for comma-bearing JSON, retained here for uniformity and for defense against a hostile declared length.

## V. Cryptographic Design

The radio is the first open path, and it is the one a key can close. The design goal is that a forged or modified frame must fail before any decision is made. Two primitives provide that property, and both are implemented in this repository with no third-party code.

### V.I. Argon2id Key Derivation

A passphrase is not a key. Argon2id (RFC 9106) [7] is a memory-hard password hash that mixes the passphrase and a salt across memory and time so that recovering the field passphrase from a captured image is expensive. The node derives a 32-byte key at initialization with the classroom profile  $t=3, p=1, m=64$  blocks (CRYPTO\_KDF\_TIME\_COST, CRYPTO\_KDF\_PARALLELISM, CRYPTO\_KDF\_MEMORY\_BLOCKS). That profile is sized to fit the RP2350 SRAM budget; it is a teaching parameter, not a hardening parameter, and the documentation says so. The salt must be at least 8 bytes; the laboratory salt is the 16 ASCII bytes `coldiron-salt-01`. The in-repo derivation is built from BLAKE2b and the Argon2 variable-length hash  $H'$ , and the RFC 9106 known-answer test runs in the Python suite.

### V.II. XChaCha20-Poly1305 per Frame

Every frame is sealed with XChaCha20-Poly1305, an AEAD that combines the ChaCha20 stream cipher and the Poly1305 one-time authenticator from RFC 8439 [6] with an extended-nonce construction. The 24-byte nonce is expanded through HChaCha20 into a per-frame subkey, which yields two properties that matter here:

- **Unpredictable nonces at scale.** A 192-bit nonce may be drawn at random for every frame from the RP2350 hardware random source, so the node never needs a shared counter that a reboot could reuse.
- **One pass for secrecy and integrity.** The same operation produces the ciphertext and a 128-bit Poly1305 tag. An attacker who guesses a valid tag succeeds with probability  $2^{-128}$ .

The associated data is the valve node identifier, a single byte (0x07 for the default node). It is authenticated but not encrypted, so a frame sealed for one node cannot be silently relabeled as another node's frame.

### V.III. Why ChaCha20 over AES on the RP2350

The RP2350 does not have a hardware AES engine; its accelerated crypto block covers SHA-256, not AES. A software AES implementation on this part is therefore both slower and riskier: table-driven AES performs data-dependent memory accesses, and those accesses create a cache-timing side channel. ChaCha20 is built only from

addition, rotation, and XOR, with no data-dependent table lookups, so it is fast in portable C and has no comparable cache-timing surface. XChaCha20-Poly1305 is thus both the modern choice and the pragmatic one for this silicon.

The primitives are split across small, independently testable modules: `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`. A constant-time comparison (`crypto_aead_tag_equal`) ensures a mismatching tag is rejected without an early-exit timing signal.

### V.IV. Key Model

Act III uses a single field key. It seals every frame on the wire and it computes the state tag over the authorization record. In the classroom build the field key is derived from one committed lab passphrase and salt, so the firmware and the gateway interoperate with no provisioning step. That is a lab convenience, not a deployment, and the documentation says so. The design keeps the roles separable so a student can reason about the real lifecycle: derive, provision per device, use, rotate on a schedule, and retire. A production build provisions key material from one-time-programmable (OTP) memory and keeps the state-tag key off the field device where possible.

## VI. Anti-Replay Window and Authenticated State

Strong AEAD is necessary and not sufficient. Two stateful controls sit above the sealed wire.

### VI.I. The Anti-Replay Window

A captured command is authentically sealed, so a controller that checks only the tag will happily apply it again. The authorization record keeps `last_seq`, the highest sequence number ever accepted, and `valve_auth_apply` accepts a command only when its sequence is strictly greater than `last_seq`. The order of checks is deliberate: the sequence test is evaluated first, then the tag is verified against the candidate record the command would produce, and only then is the record updated. A replayed valid command therefore fails on

freshness, not on cryptography, which is exactly the lesson: authentication is not freshness.

## VI.II. The Authenticated State Tag

The centerpiece is the verdict itself. The controller decides with a boolean in SRAM, call it `granted`, and an attacker with a debug probe and a GDB session does not break the cipher; they set `granted = true`. To detect that, the authorization record is nine bytes:

```
record = granted[1], seq[4], last_seq[4]
```

and the state tag is an XChaCha20-Poly1305 tag over that record, computed under the field key with a deterministic nonce built from the sequence number and the domain byte `0xA7`:

```
tag = AEAD_seal(fieldkey, nonce(seq), record, AD=0)
```

`valve_auth_state_ok` recomputes the tag and compares it in constant time, and the guarded command path requires it before the valve moves. A debugger that flips `granted` without recomputing the tag changes the record, so the stored tag no longer matches and the release is denied ahead of the actuator. The wire is authenticated, and so is the verdict.

## VI.III. TOCTOU in One Session

The two attacks are independent and teach different defaults. The window stops a valid command from working twice. The tag stops an unauthorized verdict from existing at all. Together they convert the original failure, a correct decision followed by a mutable state read (a time-of-check to time-of-use gap), into two explicit, testable checks.

## VII. Envelope Layout and Gateway Verification

The binary envelope is assembled in a fixed order and then hex-encoded:

```
envelope = nonce[24], ciphertext[L], tag[16]
```

The encoder emits lowercase hex with a trailing NUL, and the decoder accepts either case. It requires an even-length string of at least the nonce plus tag size, bounds the decoded length, recomputes the tag over the associated data and ciphertext, compares in constant time,

and only then decrypts. Any malformed, truncated, tampered, or forged envelope returns false and yields no trusted plaintext.

On the gateway side, `scripts/gateway.py` mirrors the same construction in pure Python using the standard library and the `field_crypto` module. The processing order is deliberate:

1. Parse the `+RCV` line by declared length to recover the hex envelope.
2. Authenticate and open the envelope. If the tag does not verify, log the frame as `UNAUTHENTICATED` with an empty command and stop. The forged body is never parsed.
3. Only for an authenticated frame, recover the one-byte command, check it against the guarded set, write an OK row, and answer with a sealed command carrying the next monotonic sequence and the state tag over the record the command would produce.

The CSV log therefore grows by one row per frame with columns `utc`, `sender`, `auth`, `cmd`, `rss_i_snr`, and the `auth` column is the audit trail. The spoofing client `scripts/spoof.py` holds no field key, so it cannot produce a valid envelope, and in replay mode it can only resend a captured command that the window will refuse.

## VIII. The FROSTLINE Implant

Act III is the first act with a malware track. The implant is real in technique and inert in effect, and it is confined to a single module compiled only under a build guard. This section states what it does, how it is detected, and the honest limit of the artifact.

### VIII.I. Build Guard

`src/implant.c` is compiled only when `SANDBOX_ONLY` is defined. The clean firmware build does not define it, so the shipping image contains no implant. The native test build and the companion CTF build do define it, and the test build also defines `IMPLANT_HOST MOCK`, which replaces the CoreDebug register and the reserved flash sector with controllable host variables. This is the containment boundary: the malware track is a build configuration, not a hidden runtime feature of the shipping firmware.

### VIII.II. The Covert Beacon

Every `IMPLANT_BEACON_INTERVAL_TICKS` (8) ticks, `implant_beacon` emits a frame to the local classroom hub. The frame is an 8-byte array: the 4-byte magic preamble `DE AD BE EF` (`IMPLANT_BEACON_MAGIC`), then a 4-byte synthetic status blob containing the low and high bytes of the tick counter, the marker byte `0xC7`, and a byte reporting whether a probe is attached. The beacon carries no real data and targets no external address.

### VIII.III. The Logic Bomb

`implant_handle_command` arms the bomb when an inbound frame begins with the exact 8-byte magic `FROSTLINE` (`IMPLANT_ARM_MAGIC`, the bytes 'F' 'R' 'O' 'S' 'T' 'L' 'I' 'N' 'E'). It records `g_implant_trigger_tick` as the current tick plus `IMPLANT_TRIGGER_DELAY_TICKS` (3). On the trigger tick, `implant_detonate` calls `valve_close` directly, clears the armed latch, and writes the persistence marker. The detonation bypasses the operator, the gateway, the envelope, the anti-replay window, and the state tag, because it calls the actuator underneath all of them. That is the scope lesson of the act.

### VIII.IV. The Anti-Debug Trap

Every tick, `implant_tick` calls `implant_debug_attached`, which reads the CoreDebug `DHCSR` register at `0xE000EDF0` (`VALVE_IMPLANT_DHCSR_ADDR`). Bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT` (`IMPLANT_DHCSR_DEBUGEN` and `IMPLANT_DHCSR_HALT`). When either bit is set, `implant_tick` returns early, so the beacon and the bomb are both suppressed while a probe is attached. The implant behaves benignly under observation and resumes when the probe is detached. This is the minimal anti-analysis trap, and it is deliberately simple so a student can see the branch, set a breakpoint after it, and prove the payload.

### VIII.V. The Persistence Preview

On the first detonation, `implant_persist` writes the marker byte `0xC7` (`IMPLANT_MARKER_BYTE`) to the reserved sector at `0x103FF000` (`VALVE_IMPLANT_RESERVE_ADDR`), the final sector of ex-

ternal flash, and sets `g_implant_persisted`. A second detonation does not write again. This is a deliberate preview of Act IV, where persistence is the whole lesson: a payload that leaves a trace outside its own code, so that removing the code is not obviously enough.

### VIII.VI. Detection and Neutralization

The implant is detected by image comparison: the clean build and the `SANDBOX_ONLY` build differ by the implant module and its symbols. It is detected in the beacon by its periodic `DE AD BE EF` frame. It is detected by static analysis by the arming magic and the `valve_close` call in `implant_detonate`. It is detected under GDB because the `DHCSR` read is a branch that a student can stand after. The seven native implant tests (`test_implant_init`, `test_implant_debug_attached`, `test_implant_handle_command`, `test_implant_beacon`, `test_implant_anti_debug`, `test_implant_logic_bomb`, and `test_implant_persist_once`) assert each behavior and its containment.

Neutralization is not a one-byte patch. It is the removal of the code path and the build flag: build without `SANDBOX_ONLY`, verify the flashed image against a signed digest, and route every actuator move through the guarded, authorized command path. In the lab, the anti-debug trap is defeated by understanding the branch, not by hiding from it.

### VIII.VII. Honest Limitation

The implant is a benign educational implant. It is confined to the breadboard, guarded by `SANDBOX_ONLY`, and has no network. It actuates only the student's own servo and writes only to a reserved sector that holds nothing else. It is a demonstration of technique, not tradecraft: it does not encrypt itself, it does not load a second stage, and it does not persist across a reflash, which is precisely what Act IV will add. The value of the exercise is that it makes the scope limit of an authenticated protocol concrete, not that it simulates a real-world implant.

## IX. Artifact Contract

Provisioning constants are stored in a JSON artifact:

```
{
  "format": "pipeline-valve-packets-demo-v1",
  "frame_version": 1,
  "node_id": 7,
  "hub_address_hex": "0001",
  "frame_size": 48,
  "estop_wait_ms": 5000,
  "servo_close_pulse_us": 500,
  "servo_open_pulse_us": 1500,
  "dht_timeout_us": 240,
  "lcd_i2c_address_hex": "27",
  "max_rcv_len": 256,
  "example_frame": "{\"cmd\":\"open\\\", \"seq\":1}"
}
```

`scripts/gen_packet.py` emits `include/packet_artifact.h` from the JSON byte-for-byte. The CMake build regenerates the header before compiling and fails when the committed header is stale, so firmware constants and the test suite always read the same provisioning data. The receive limit is 256 bytes, matching the radio command and receive buffers so a maximum-size hex envelope fits with framing headroom.

## X. Fail-Closed Policy and ESTOP Priority

A valve has a safe state, and the controller must choose it deliberately. The valve is **fail-closed**: `valve_init` seats it closed at boot, `valve_fail_closed` drives it closed and latches a fault, and the monitor calls it on link loss. A lost gateway link for longer than `VALVE_ESTOP_WAIT_MS` leaves the valve closed, because a command that cannot be authorized must not be assumed.

The emergency stop is the highest-priority input. `monitor_service_inputs` services the ESTOP button before it polls the operator remote or the radio, and `monitor_apply_ir_command` refuses to act while the latch is set. A one-bit safety input therefore outranks every authenticated byte on the air. The lesson is that fail mode and priority are policy choices, and naming them is part of the design.

## XI. Attack Exercises and Hardening

The classroom runs the malware track and the fix track against the same build.

### XI.I. Malware Track: The Beacon, the Bomb, and the Trap

Students flash the `SANDBOX_ONLY` image, locate the `DE AD BE EF` beacon and its 8-tick interval, inject the `FROSTLINE` magic to arm the bomb, and watch the valve close three ticks later with no operator and no gateway. They then attach a probe and observe that the implant suppresses itself, break after the `DHCSR` check, and prove the payload with the trap bypassed. They finish by reading the `0xC7` marker in the reserved sector and explaining why it is a preview of persistence. The centerpiece is the scope claim in one session: a correct, sealed command path does not protect the actuator from code that shares the chip.

### XI.II. Red Half: Forgery and Replay

A structurally plausible command with a random nonce and a random tag is injected with `scripts/spoof.py --mode bad-tag`. The spoof tool holds no field key, so the tag cannot verify, and the controller denies before parsing any body. A captured command is replayed with `--mode replay`; the sequence is not greater than `last_seq`, the command fails on freshness, and the fault lamp lights. This is a pedagogical reintroduction of a well-known link failure mode: at the physical and MAC layer nothing binds a frame to a physical transceiver, so authentication must live in the payload.

### XI.III. Red Half: The Verdict in SRAM

The exercise halts the controller under the Debug Probe, breaks in the valve authorization path, sets `granted = true`, and continues. Under a controller that trusts the boolean the valve moves. Under the Act III controller the state tag is recomputed over the modified record, the mismatch is found, and the command is denied ahead of the actuator.

### XI.IV. Blue Half: Sealing It

The blue-half controls map one-to-one onto the red-half and malware findings:

- **Sealed command path.** Open the envelope under the field key, guard the command byte against the open and close set, verify the sequence and the state tag.

- **Anti-replay window.** `last_seq` and a strictly monotonic sequence rule.
- **Authenticated state tag.** A keyed tag over the nine-byte authorization record, computed with a domain-separated nonce and verified in constant time.
- **ESTOP priority.** Latch first, refuse remote commands while latched.
- **Fail closed.** Seat the valve at boot, on link loss, and on every fault.
- **Build integrity.** Do not define `SANDBOX_ONLY` in production, and sign and verify the firmware image.
- **Debug lockdown.** On the deployed part, burn secure-boot and debug-disable in OTP so SWD cannot read or write SRAM.

### *XI. V. What the Hardening Buys, and What It Does Not*

The command path closes the forgery, replay, and verdict-tamper surfaces:

- A forged or modified frame fails the tag before parsing.
- A captured valid command fails on freshness on second use.
- A debugger-written verdict fails the state-tag check before the valve moves.
- The valve node identity is bound into the associated data, so a frame cannot be relabeled for another node.

It does not, by itself, remove an implant that already runs on the device. That is a build-integrity and supply-chain control, not a protocol control, and it is the central lesson of the act.

## **XII. Implementation Compliance Mapping**

The repository implements the full classroom loop:

- **Peripherals and control:** `src/monitor.c` drives the tick and the valve policy; `src/valve.c` sequences the actuator and fails closed; `src/sensor.c` samples the DHT11 process band; `src/display.c` renders the SCADA readout; `src/status_led.c` maps the verdict to the red, yellow, and green lamps; `src/button.c` debounces the emergency stop; `src/servo.c` drives the valve PWM; `src/ir_remote.c` decodes NEC operator commands.
- **Command and state:** `src/control.c` opens and applies sealed commands with a guarded command set; `src/valve_auth.c` holds the authorization record, the monotonic anti-replay window, and the authenticated state tag.
- **Malware:** `src/implant.c` implements the `SANDBOX_ONLY` beacon, logic bomb, CoreDebug anti-debug trap, and reserved-sector persistence marker.
- **Radio:** `src/radio.c` provisions the transceiver, builds AT+SEND, parses +RCV with the declared-length discipline, and pumps CRLF lines into 256-byte buffers.
- **Cryptography:** `src/chacha20.c`, `src/poly1305.c`, `src/crypto_aead.c`, `src/blake2b.c`, `src/argon2.c`, `src/crypto_kdf.c`, and `src/envelope.c`, with `include/field_secrets.h` holding the lab-only key material.
- **Tooling:** `scripts/gen_packet.py`, `run_tests.py`, `check_coverage.py`, `audit_c_standard.py`, `audit_python_standard.py`, and `gen_banner.py`.
- **Classroom:** `scripts/gateway.py` (gateway provisioning, authentication, CSV logging, sealed command replies), `scripts/spoof.py`, `scripts/sim_edge.py`, and the pure-Python interoperable crypto in `scripts/field_crypto.py`.
- **Tests:** 132 native C cases and 409 checks with 0 failures. They cover the full DHT waveform and every timeout shape, the valve state machine and its bounded travel, the sealed command path and its guards, the authorization window and state tag, the emergency-stop priority, fail-closed on link loss, the declared-length parser, the servo and LED mappings, the operator remote paths, the implant beacon, logic bomb, anti-debug, and write-once persistence, and the cryptographic primitives against published vectors.

The tests run natively on the host via mock Pico SDK headers, reaching 100% line coverage on `crc.c`, `sensor.c`, `display.c`, `radio.c`, `status_led.c`, `button.c`, `servo.c`, `ir_remote.c`, `valve.c`, `control.c`, `valve_auth.c`, `chacha20.c`, `poly1305.c`, `crypto_aead.c`, `blake2b.c`, `argon2.c`, `crypto_kdf.c`, `envelope.c`, `monitor.c`, and `implant.c` under LLVM source coverage, for 1982 / 1982 lines. `main.c` is excluded from coverage by design.

### XIII. Threat Model and Limitations

The security claims of this build are bounded and stated plainly.

- **Lab key profile.** Argon2id runs at  $t=3$ ,  $p=1$ ,  $m=64$  blocks so the derivation fits the RP2350 SRAM budget. This is weaker than a production password-hashing profile and must be raised on a host gateway.
- **Keys in flash are development-only.** `include/field_secrets.h` commits a shared passphrase and salt so the firmware and the Python gateway derive the same key in the classroom. Production firmware must provision key material from OTP memory at manufacture and must never embed a passphrase, salt, or derived key in flash.
- **Open debug port.** The Debug Probe is the instrument for both the malware analysis and the verdict-tamper exercise. The authenticated state tag makes a tampered verdict detectable, but a probe that can read the field key from SRAM defeats the design. Production must disable debug in OTP.
- **Replay scope.** The window rejects a replayed command, but a reboot resets `last_seq` to zero. A command captured before a reboot can therefore be replayed after one. A production controller persists the sequence floor in non-volatile memory.
- **The implant is benign and guarded.** It is compiled only under `SANDBOX_ONLY`, confined to the breadboard, and has no network. It touches only its own beacon frame, its arming latch, the servo close path, and the reserved sector.
- **The implant bypasses the protocol entirely.** No amount of wire authentication stops a module that calls the actuator directly. Mitigating that is a build-integrity, signing, and debug-lockdown problem, not a protocol problem.
- **Fail mode trade-off.** The valve is fail-closed and the emergency stop has absolute priority. Any change to either must be a policy decision, not a code accident.
- **Infrared path unauthenticated.** The NEC operator remote has no key and no anti-replay state. Any compatible remote can send a command. The sealed radio path is the authorization path; the optical surface is a documented exposure.
- **Sensor trust boundary.** The DHT11 is a check-summed but not authenticated one-wire sensor; the

process band is only as trustworthy as the physical wiring and the edge timing.

- **RSSI and SNR are informational.** Neither is a reliable origin indicator.
- **Artifact guardrail.** The build-time artifact check verifies provisioning consistency, not security.
- **Denial of service.** An attacker on the band can still jam or flood the receiver; authentication is not availability.

#### XIII.I. Future Work

- Provision the field key from RP2350 OTP memory and add a documented rotation procedure.
- Persist the anti-replay sequence floor in non-volatile memory so a reboot does not reset freshness.
- Add a signed-image verification step to the flash procedure and a measured boot chain on the RP2350.
- Burn debug-disable and secure-boot settings in OTP for the deployed part.
- Add an actuator-state ledger and an interlock alarm debounce.
- Raise the Argon2id profile on the gateway and record the derivation cost as a measured parameter.
- Extend the malware track toward the Act IV persistence lesson with a controlled reflash-resistant marker and its removal procedure.

### XIV. Conclusion

OPERATION IRON VEIN turns a trusting valve controller into a defensible one, and then shows why the defended protocol is not the whole story. The controller drives the full Embedded Hacking peripheral set, so a state failure has a visible and physical consequence at the valve. The LoRa command path is sealed end to end with XChaCha20-Poly1305 keyed through Argon2id, implemented and tested entirely in-repo, with the valve node identity bound as associated data. Act III adds a guarded command set, a monotonic anti-replay window so a captured command dies on second use, and an authenticated state tag so a debugger-written verdict dies before the actuator moves. The emergency stop outranks every authenticated byte, and the valve fails closed on loss of authority. The gateway authenticates before it parses, so the spoofing client that once forged a command now fails at the tag, and the replay that once reopened a line now fails at the window. And then

there is the implant: a benign, `SANDBOX_ONLY` FROST-LINE module that beacons, arms, hides from a probe, and closes the valve without ever touching the sealed wire. The firmware, gateway toolset, artifact guardrail, and 100%-line-covered native test suite provide a reproducible baseline, and the threat model states exactly which assumptions remain. That combination, a sealed command path next to an honest account of the module that ignores it, is the lesson Act III owes the story: the protocol was never the hard part. The code on the chip was.

## XV. References

- [1] D-Robotics, “DHT11 Digital temperature and humidity sensor datasheet,” Aosong Electronics Co., Ltd, 2010.
- [2] Vishay Semiconductors, “IR Receiver Modules for Remote Control Systems (VS1838B),” Vishay Intertechnology, datasheet 81910, 2018.
- [3] Anonymous the Security Researcher, “Analysis of serial-AT sub-GHz radios: cleartext configuration and absent frame authentication,” Embedded security working notes, 2022.
- [4] R. Menon and A. Prakash, “On the (in)security of LoRa point-to-point links under address spoofing,” *ACM SIGCOMM Embedded Systems Workshop*, 2023, pp. 12-19.
- [5] K. Thomas, “The reverse engineering self-study course,” <https://github.com/mytechnotalent/Reverse-Engineering>, 2026.
- [6] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” RFC 8439, Internet Engineering Task Force, June 2018.
- [7] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, September 2021.
- [8] A. Costin and J. Zaddach, “A large-scale analysis of the security of embedded firmwares,” *Proceedings of the 23rd USENIX Security Symposium*, 2014, pp. 95-110.